

Informe técnico de arquitectura de la SPA

Módulo Clientes integrado a PharmaSoft. Autor: Del Carpio. Rama de trabajo: **feature/clientes-delcarpio**. El informe explica el código actual; el anexo independiente conserva las capturas originales de los once casos A7.

B1. Estructura del proyecto

```
src/app/
  core/
    config/menu.ts
    models/{error-response,pagina-response}.ts
    utils/http-error.ts
  shared/pages/no-encontrado/
  layout/{main-layout,header,sidebar}/
  features/
    inicio/
    categorias/
      models/ services/ pages/{categoria-list,categoria-form}/
      categorias.routes.ts
    clientes/
      models/ services/ pages/{cliente-list,cliente-form}/
      clientes.routes.ts
  app.ts app.config.ts app.routes.ts
```

Árbol resumido de carpetas reales: se omiten estilos, plantillas y pruebas para facilitar la lectura. **core** concentra configuración, contratos y utilidades transversales; **shared** aloja presentación reutilizable, actualmente la página 404; **layout** compone la estructura persistente; **features** agrupa cada dominio con sus modelos, servicio, páginas y rutas. Clientes reproduce la organización de Categorías.

B2. Mapa completo de rutas

URL	Componente / padre	Carga	Título
/	Redirección bajo MainLayout	redirectTo inicio	Hereda Inicio
/inicio	Inicio / MainLayout	loadComponent	Inicio
/categorias	CategoriaList / MainLayout	loadChildren del feature	Categorías
/categorias/nuevo	CategoriaForm / MainLayout	Dentro del feature diferido	Nueva categoría
/categorias/:id/editar	CategoriaForm / MainLayout	Dentro del feature diferido	Editar categoría
/clientes	ClienteList / MainLayout	loadChildren del feature	Clientes
/clientes/nuevo	ClienteForm / MainLayout	Dentro del feature diferido	Nuevo cliente
/clientes/:id/editar	ClienteForm / MainLayout	Dentro del feature diferido	Editar cliente
/**	NoEncontrado / sin layout padre	loadComponent	Página no encontrada

Fuentes: src/app/app.routes.ts:4-33; features/clientes/clientes.routes.ts:5-9; features/categorias/categorias.routes.ts:5-9. Los componentes de cada feature se importan en su archivo de rutas; no se difieren individualmente.

B3. Separación de responsabilidades

Pieza / tipo	Responsabilidad	Qué no hace
ClienteService / servicio	Concentra cinco operaciones HTTP; HttpParams y Observables tipados.	No dibuja alertas, valida formularios ni navega.
ClienteList / componente	Signals de página, tamaño, orden y estados; filtro computed; confirmación y recarga tras baja.	No construye URLs ni llama HttpClient directamente.
ClienteForm / componente	Validadores, carga por ID, normalización de opcionales, creación/edición y navegación.	No consulta Oracle ni decide unicidad de DNI/correo.
CategoriaService / servicio	CRUD tipado sobre environment.apiUrl/categorias.	No conserva estado visual ni presenta errores.
CategoriaList / componente	Carga y filtro local, estados de pantalla y eliminación con mensaje contextual.	No implementa transporte HTTP ni reglas de integridad SQL.
CategoriaForm / componente	Formulario reactivo, precarga, validación y guardar/navegar.	No persiste directamente ni gestiona rutas globales.
MainLayout / componente	Compone Header, Sidebar y router-outlet; alterna menú colapsado.	No contiene CRUD ni solicitudes de datos del dominio.
Header / componente	Identidad visual, enlaces superiores y evento toggleMenu.	No carga clientes ni define opciones del sidebar.
Sidebar / componente	Renderiza MENU con RouterLink y RouterLinkActive.	No define endpoints ni administra permisos de backend.

Evidencia del código

```

ClienteService (services/cliente-service.ts:8-19):
  private readonly http = inject(HttpClient);
  private readonly url = `${environment.apiUrl}/clientes`;
  return this.http.get<PaginaResponse<Cliente>>(this.url, { params });

```

```

ClienteList (pages/cliente-list/cliente-list.ts:29-34):
  const texto = this.filtro().trim().toLowerCase();
  return (this.respuesta()?.contenido ?? []).filter(c => ...);

```

```

MainLayout (layout/main-layout/main-layout.html:2-5):
  <app-header ... (toggleMenu)="alternarMenu()" />
  <app-sidebar ... />
  <main ...><router-outlet /></main>

```

Las rutas de archivos de los fragments se resuelven desde src/app/features/clientes y src/app respectivamente. Categorías usa el mismo patrón de servicio/página, con un arreglo simple; Clientes adapta la presentación a PaginaResponse<Cliente> y a la baja lógica.

B4. Flujo de Registrar cliente

Recorrido verificado en `ClienteForm.guardar()`, `ClienteService.crear()` y `ClienteList.cargar()`. Los números remiten al código propio, no a una implementación propuesta.

Paso	Operación y estado
1. Clic Registrar	<code>ngSubmit</code> llama <code>guardar()</code> ; <code>ClienteForm</code> limpia espacios y valida. Si es inválido, marca campos <code>touched</code> y no envía HTTP.
2. DTO y signal	<code>telefono/direccion</code> vacíos se convierten en <code>null</code> ; <code>guardando.set(true)</code> evita envíos duplicados.
3. Servicio y HTTP	<code>ClienteService.crear(dto)</code> devuelve <code>Observable<Cliente></code> ; <code>HttpClient</code> envía <code>POST /api/v1/clientes</code> .
4. Backend	Controller valida DTO; servicio comprueba unicidad y guarda mediante <code>repository/JPA</code> en Oracle.
5a. Respuesta 201	Callback <code>next</code> navega a <code>/clientes</code> ; <code>finalize</code> devuelve <code>guardando</code> a <code>false</code> .
6a. Nueva lista	<code>ClienteList.ngOnInit</code> llama <code>cargar()</code> ; GET paginado 200; <code>respuesta.set(datos)</code> , <code>pagina.set(datos.pagina)</code> , <code>cargando=false</code> ; la tabla muestra contenido.
5b. Respuesta 409	Callback <code>error</code> conserva el formulario y fija <code>error.set(mensajeError(err))</code> ; <code>finalize</code> devuelve <code>guardando</code> a <code>false</code> . No navega ni crea una nueva fila.

Diagrama de secuencia

```

Registrar
|
ClienteForm.guardar() -- inválido --> campos touched / sin POST
| válido: DTO + guardando=true
ClienteService.crear(dto)
| POST /api/v1/clientes
PharmaBackend -> ClienteServiceImpl -> JPA -> Oracle
|
+-- 201 --> next -> Router /clientes
|               -> ClienteList -> GET 200
|               -> respuesta.set(datos) -> tabla
|
+-- 409 --> error.set(mensajeError(err)) -> alerta en formulario

```

En ambas respuestas: `finalize -> guardando=false`

Fuentes: `features/clientes/pages/cliente-form/cliente-form.ts:62-79`; `services/cliente-service.ts:26-28`; `pages/cliente-list/cliente-list.ts:37-59`. Backend: `ClienteServiceImpl.create` comprueba DNI/correo antes de guardar. Las capturas A5 y A6 muestran respectivamente POST 201 y POST 409.

B5. Revisión de decisiones (1-4)

1. Archivos existentes modificados

La integración del feature modifica **dos archivos existentes**: `src/app/app.routes.ts:15-19` agrega la entrada diferida y `src/app/core/config/menu.ts:10` agrega Clientes. `PaginaResponse`, las páginas, modelos, servicio, rutas y pruebas de Clientes son archivos nuevos; `app.routes.spec.ts` también es nuevo. Este conteo corresponde a la integración frontend de Clientes, no a reparaciones anteriores de Categorías ni a cambios del backend. El reducido número de puntos de integración muestra que el dominio está encapsulado.

2. Listado sin HttpClient directo

`ClienteList` inyecta `ClienteService` (`cliente-list.ts:18`) y usa `listar()` en la línea 42; el servicio concentra `HttpClient` y la URL (`cliente-service.ts:8-10`). Si cambia el host o prefijo común, se ajusta `environment.apiUrl`; si cambia el recurso o contrato, se adapta el servicio/modelo. El listado conserva su responsabilidad visual y no repite configuración HTTP.

3. Ventaja y verificación de loadChildren

`app.routes.ts:17-18` usa `import()` para cargar `CLIENTES_ROUTES` al entrar al dominio. Sus páginas quedan fuera del punto de entrada inicial de rutas y en el chunk diferido del feature. No implica un chunk independiente por cada formulario. La compilación `npm run build` terminó correctamente y produjo esta línea real:

```
chunk-5ZZDPHEZ.js | clientes-routes | 13.77 kB | 4.27 kB
```

La última columna es el tamaño estimado de transferencia; el hash puede variar en futuras compilaciones. El output separa explícitamente Initial chunk files y Lazy chunk files.

4. Persistencia del encabezado y sidebar

Categorías y Clientes son hijos del mismo `MainLayout` (`app.routes.ts:6-24`). Header y Sidebar están fuera del router-outlet interno (`main-layout.html:2-5`), así que el cambio entre esos hijos sustituye la página del dominio y conserva las instancias del layout. Esto no significa ausencia total de detección de cambios: `RouterLinkActive` actualiza el resaltado. La ruta global 404 queda fuera del layout. `app.routes.spec.ts` verifica la navegación integrada y la persistencia del shell.

B5. Revisión de decisiones (5-7) y entrega

5. Visibilidad de Clientes según usuario

El punto central para declarar permisos o roles del menú sería `src/app/core/config/menu.ts:1-11`. Sidebar consume ese arreglo (`layout/sidebar/sidebar.ts:3,12`), evitando duplicar opciones en plantillas. Actualmente no existe filtrado por usuario: habría que extender `Menuitem` y derivar el menú visible desde un servicio de sesión. Una vez implementado ese mecanismo, cambiar la política de una opción bastaría en `MENU`. Ocultar el enlace no autoriza ni bloquea el acceso; guardas de ruta y backend deben aplicar el permiso real.

6. Duplicación y posible reutilización

`CategoriaService` y `ClienteService` repiten obtener/crear/actualizar/eliminar (`categoria-service.ts:18-31`; `cliente-service.ts:22-36`). Los listados repiten carga, error, confirmación y filtro; los formularios repiten guardar y navegar (`categoria-form.ts:48-68`; `cliente-form.ts:62-79`). Se puede extraer un adaptador CRUD genérico para las operaciones comunes y componentes pequeños de alerta/estado de carga. Conviene mantener la paginación y la baja lógica en Clientes, evitando que una abstracción imponga comportamiento incorrecto a Categorías.

7. Alcance de búsqueda y alternativa global

El computed filtra solamente `respuesta().contenido` (`cliente-list.ts:29-34`), es decir, la página que ya llegó por GET. No consulta registros de otras páginas y no genera HTTP al escribir. Para búsqueda global, el backend debe aceptar un parámetro `q` y filtrar antes de paginar/contar; `ClienteService.listar` debe añadir `q` a `HttpParams` (`cliente-service.ts:12-18`). El listado debe volver a página 0, consultar con `debounce/cancelación` y usar los totales del resultado filtrado. La evidencia A8 muestra el filtro local sin solicitudes.

Estado de validación y entrega

Compilación production verificada: exit 0. Revisión final: 45 pruebas aprobadas en 14 archivos (`ng test --watch=false`). Capturas A1-A11 reunidas en docs/evidencias y en **anexo-evidencias.pdf**. A7 registra teléfono 987654323 para ID 43. A9-A10 muestran 204/200/409 y fila Inactivo; no muestran Method ni Headers completos, limitación conservada en el anexo.

El informe técnico de cinco páginas queda en docs/informe-arquitectura.pdf. DelCarpio_LP2_S7_Autonomia.pdf reúne ese informe y el anexo de 16 páginas para subir un solo archivo al aula (21 páginas con anexos). El anexo también se conserva por separado. El usuario indicó que el docente eximió distribuir commits en días distintos; no se alteran fechas ni se inventa historial.

PR hacia develop: pendiente de crear. Su enlace deberá incorporarse a esta sección después del paso de versionamiento. Este documento no afirma que ya se hayan realizado tres commits, publicado la rama o entregado al aula.